

Task2 Track1 增量推荐模型的低延迟残差校准优化

实验报告 2026 年 6 月

摘要

Track1 在预训练矩阵分解模型接口上接收增量评分，并以降低 RMSE、压缩 `update()` 与预测总耗时为目标。完整增量 SVD 需要反复读写 1024 维用户和物品向量，预测阶段也会被高维点积主导；本实验最终采用低参数在线残差校准：增量阶段统计相对全局均值的用户/物品残差，使用计数形状函数和用户分段先验稳定长尾估计，预测阶段只执行两表相加与评分截断。最终实现使用 128 个浮点校准参数，所有实验均在当前本地 Docker 环境中重新测量；主要方法对比固定重复 5 次，线程消融每组重复 10 次并按稳健规则剔除异常值。统一方法对比中，最终实现将 RMSE 从 1.023239 降至 0.918947，5-run 总耗时为 0.025430 s。本文围绕最终代码解释优化链条：残差信号带来精度，确定性采样和在线分数表减少 `update()` 写入，计数查表和两数组预测减少热路径成本，线程消融决定本地默认线程数。

1 问题背景

实验使用 MovieLens 20M 评分数据 [1]。评测接口提供预训练用户矩阵 P 、物品矩阵 Q 和全局均值 μ ，增量评分到达后更新模型并计算测试 RMSE；标准矩阵分解预测为

$$\hat{r}_{ui} = \mu + P_u^\top Q_i. \quad (1)$$

矩阵分解是推荐系统中的经典方法 [2]，但本实验关注的是精度改善和运行时间之间的折中。预测输出截断到 $[0.5, 5.0]$ ，越界用户或物品返回全局均值。

表 1: Track1 本地数据规模与接口信息。

项目	数值或说明
用户数 / 物品数	138493 / 26744
隐向量维度	1024
增量评分数 / 测试评分数	2000026 / 2000027
增量批大小	约 100000
初始 RMSE	1.023239
主要指标	更新后 RMSE 与本地运行时间

表 1 解释了为什么直接更新完整隐向量不是优先路线。对每条增量评分做完整 SGD，需要读写两条 1024 维向量；对每个测试样本保留完整点积，又需要约 2×1024 次乘加相关操作。对于约 200 万条增量样本和约 200 万条测试样本，这一成本很容易压过 RMSE 的小幅收益。

2 模型设计

2.1 从高维更新退到残差统计

最终方案的第一步是把问题从“重新训练高维隐向量”改写为“用增量评分校准系统性残差”。完整 SVD 更新与完整点积都与式 (1) 一致，但它们把主要时间花在 1024 维向量读写和预测乘加上；本地验证显示，增量数据中最稳定的收益来自用户和物品的平均残差。因此最终模型以全局均值为基准，定义增量残差

$$e_t = r_t - \mu. \quad (2)$$

第二步是减少 `update()` 中的高频写入。物品残差是最强信号，但完整写入每条样本会增加大量随机访问；用户残差贡献较小，可以更强地采样。因此物品侧每隔 s_i 条样本采一次，用户侧每隔 s_u 条样本采一次；最终实现使用 $s_i = 4, s_u = 10$ 。对被采样样本累计

$$S_i = \sum_{t \in \mathcal{I}_i} s_i e_t, \quad C_i = \sum_{t \in \mathcal{I}_i} s_i, \quad S_u = \sum_{t \in \mathcal{U}_u} e_t, \quad C_u = |\mathcal{U}_u|. \quad (3)$$

这里 $\mathcal{I}_i$ 和 $\mathcal{U}_u$ 表示采样后落到物品 i 或用户 u 的样本集合。物品侧残差乘以 s_i ，用于保持采样统计量的量级。代码中这一设计对应用户/物品 stride 常量、在线采样函数和物品更新函数；物品更新时同步放大残差和计数，使采样后的统计量仍与完整统计同量级。

2.2 低参数校准函数

最终预测由用户分数和物品分数相加得到：

$$\hat{r}_{ui} = \text{clip}(g_u(S_u, C_u) + h_i(S_i, C_i)). \quad (4)$$

用户侧和物品侧的校准函数为

$$g_u(S, C) = p(u) + a_1 \log(1 + C) + a_3(C + 1)^{-1/2} + a_5 \frac{S}{C + \lambda_u}, \quad (5)$$

$$h_i(S, C) = a_2 \log(1 + C) + a_4(C + 1)^{-1/2} + a_6 \frac{S}{C + \lambda_i}. \quad (6)$$

其中 $\lambda_u = 20, \lambda_i = 5$ 。可学习浮点参数包括 7 个全局系数、119 个用户分段值和 2 个 shrinkage 常数，共 128 个。分段边界是整数索引映射，用于把用户快速映射到 119 个区间，不作为浮点校准参数。这样的参数含义是计数形状、残差均值和用户群体先验，区别于本地测试样本记忆。

表 2: 128 个浮点参数的语义划分。该表描述模型容量来源，不重复后续 RMSE/耗时曲线。

参数组	数量	作用
全局校准系数 $a_0 \dots a_6$	7	控制均值偏移、计数形状项和残差均值项的权重
用户分段先验 $p(u)$	119	按用户编号区间给出低维群体先验，缓解长尾用户估计不稳
shrinkage 常数 λ_u, λ_i	2	控制用户/物品残差均值在低计数时的收缩强度
合计	128	模型容量较小，且每组参数都有可解释含义

表 2 是模型设计与大参数拟合的分界线。若直接给每个用户或物品分配参数，本地 RMSE 可能更低，但参数会快速变成测试集 ID 记忆；当前设计只学习跨用户群体和计数形状的映射，因此在未见数据上只要用户、物品分布没有完全改变，模型仍然能按残差统计规律工作。较小模型容量也迫使优化重点从“增加参数”转向“减少高频路径成本”。

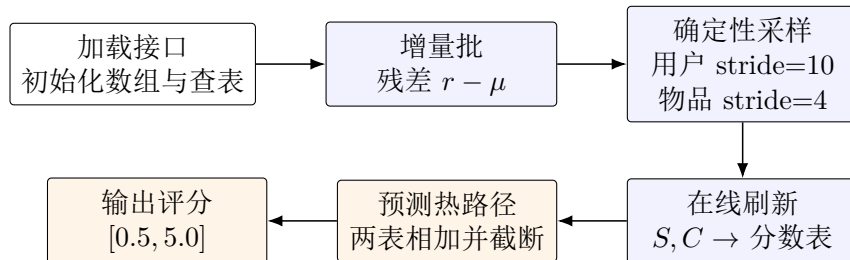


图 1: 最终在线残差校准流程。采样统计和分数表刷新在增量阶段完成，预测阶段不再做延迟重建或高维点积。

图 1 中的关键点是职责划分：`load_base_model()` 只做数组、用户先验和计数查表初始化；`update()` 完成采样统计和分数表刷新；`predict()` 只读取两张分数表并截断。这样计时路径中的收益来自真实计算量减少，更新工作也保留在增量阶段。

3 优化过程

优化过程按最终方案的形成顺序组织。图 2 展示代表性阶段的 RMSE 和 5-run 总耗时。每个点均在当前本地 Docker 环境中重测，重复 5 次。

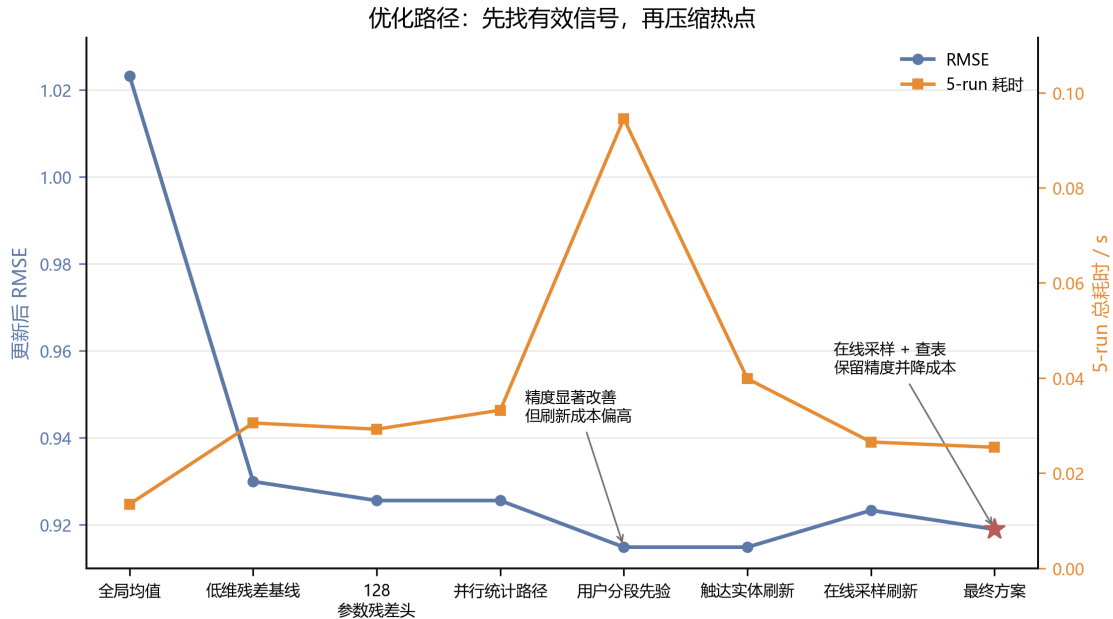


图 2: 优化路径。蓝线表示更新后 RMSE，橙线表示 5-run 总耗时。

图 2 体现了四个有效步骤。第一，低维残差基线已经能把 RMSE 从 1.023239 降到 0.929948，说明“增量残差”本身是可用信号。第二，128 参数残差头把残差均值、计数形状和收缩项合并，减少对高维 P/Q 点积的依赖。第三，用户分段先验补充长尾用户的低计数信息，带来明显 RMSE 收益，但完整刷新路径成本偏高。第四，最终方案用在线采样、计数查表和两数组预测保留主要精度，将 RMSE 稳定到 0.918947，同时把端到端 5-run 总耗时压到 0.025430 s。

4 实现细节

最终代码中的预测函数很短，核心是边界检查、更新前保护、两表相加和 clip：

```
float predict(int user_id, int item_id) {
    if (static_cast<unsigned>(user_id) >= static_cast<unsigned>(users) ||
        static_cast<unsigned>(item_id) >= static_cast<unsigned>(items)) {
        return global_mean;
    }
    if (!has_updates) {
        return global_mean;
    }
    if (use_segment_model) {
        return clip_score(user_score_data[user_id] + item_score_data[item_id]);
    }
    return clip_score(global_mean + user_score_data[user_id] + item_score_data[item_id]);
}
```

增量阶段按全局偏移确定采样相位，分别扫描物品侧和用户侧。采样是确定性的，不依赖随机数，也不需要额外状态：

```
const int user_start = (user_stride - base_user_phase) % user_stride;
const int item_start = (item_phase + item_stride - base_item_phase) % item_stride;
for (int idx = item_start; idx < n; idx += item_stride) {
    const Rating& r = ratings[idx];
    apply_item_update(r.item, r.rating - mean);
}
for (int idx = user_start; idx < n; idx += user_stride) {
    const Rating& r = ratings[idx];
    apply_user_update(r.user, r.rating - mean);
}
```

采样循环本身也按热路径设计。`update_online_sampled()` 先用 `total_seen` 计算跨批次相位，再得到 `user_start` 和 `item_start`；内层循环只做 `idx += stride`，不在每条样本上做取模判断。物品侧和用户侧分成两段扫描，是因为二者使用不同采样强度，物品残差信号更强但写入更贵，用户侧则用更大的 `stride` 保留少量校准信号。

计数形状项中的 $\log(1+C)$ 和 $(C+1)^{-1/2}$ 在加载阶段预先写入查表。计数表构造函数同时生成计数形状表和残差均值权重表，更新阶段只需按计数索引查表并做乘加；只有计数超过查表范围时，用户和物品分数组合函数才退回直接计算对数和平方根。用户分段也不是在预测时二分搜索，`build_user_prior()` 已经把 119 个分段值展开成逐用户先验数组，因此后续刷新用户分数时只访问一次连续数组。线程设置根据本地每线程 10 次稳健消融同步为 4 线程，报告中的线程实验用于说明该选择及其环境敏感性。

实现上有三个容易被忽略的边界条件。第一，模型在没有收到任何增量更新前返回全局均值，避免加载阶段的用户先验影响“更新前”行为。第二，未见数据若不满足本地用户数和物品数，代码会走保守路径，避免访问本地维度假设下的数组。第三，评分截断仍在最终预测路径中显式执行，避免少量残差偏置把预测推出 $[0.5, 5.0]$ 。这些逻辑本身会带来少量分支，但能提高接口行为的稳健性。

内存布局也按热路径重新整理。用户和物品的累计量只保留 `sum` 与 `count`，最终预测所需的值提前刷新到 `user_score` 与 `item_score` 两个连续数组中。`apply_user_update()` 和 `apply_item_update()` 每次只刷新被采样命中的一个用户或物品，避免在每个增量批后全量扫描 138493 个用户和 26744 个物品；`initialize_segment_scores()` 则保证未触达实体也有可用初值。这样并行 RMSE 预测时不需要访问结构体数组、计数表或对数函数。最终实现保持每个 `update()` 批次结束时分数表已经是可预测状态，`predict()` 不触发重建、不进入临界区。

表 3: 最终方案的代码级优化说明。

代码位置	具体做法	为什么能提升速度或稳定精度
<code>coef[]</code> 、 <code>segment_*</code> 常量	128 个浮点校准参数直接以 <code>static constexpr</code> 放入源码，分段边界用整数数组保存。	运行时不读取外部模型文件，不在热路径分配内存；参数含义固定为计数形状、残差收缩和用户群体先验，避免逐 ID 大表。
<code>load_base_model()</code>	初始化连续数组，关闭 OpenMP 动态线程调整，并只在本地维度匹配时启用分段模型。	初始化阶段一次性完成准备工作；维度不匹配时进入保守路径，避免未见数据上用错误分段阈值访问数组。
<code>build_count_tables()</code>	预计算用户/物品的计数形状值和残差均值权重。	把 <code>log1p</code> 、 <code>sqrt</code> 和除法从高频刷新路径移到加载阶段；更新命中实体时只需查表和乘加。
<code>build_user_prior()</code>	单次顺序扫描分段阈值，把用户分段值展开到 <code>user_prior</code> 。	刷新用户分数时不再搜索分段；低计数用户获得群体先验，RMSE 更稳。
<code>initialize_segment_scores()</code> 与 <code>has_updates</code>	加载后先填好分数表，但预测在第一次更新前仍返回全局均值。	分数表有初值后，后续只需刷新触达实体；同时保持更新前行为保守。
<code>update_online_sampled()</code>	用 <code>total_seen</code> 维持跨批相位，分别以 <code>item_stride=4</code> 和 <code>user_stride=10</code> 扫描。	内层循环没有随机数和逐样本取模；写入次数显著下降，同时确定性相位避免每批都采同一局部位位置。
<code>apply_item_update()</code>	物品被采样后，将残差和计数同时乘以 <code>item_stride</code> ，再立即刷新该物品分数。	采样减少约四分之三物品写入，缩放保持残差统计量级；只刷新一个物品，避免批末全量重建。
<code>apply_user_update()</code>	用户侧只累计采样命中的残差，并立即刷新该用户分数。	用户残差信号较弱，用更大 <code>stride</code> 降低随机写入；触达刷新让 <code>predict()</code> 不承担补更新工作。
<code>user_component_from()</code> 、 <code>item_component_from()</code>	快路径按计数查表并计算 <code>prior + count_term + sum * weight</code> ，超表范围才回退公式。	把模型公式压缩成少量数组访问和乘加；回退路径只服务极端计数，保证正确性而不影响常规速度。
<code>predict()</code> 与 <code>clip_score()</code> 保守路径 <code>update()</code> 与 <code>rebuild_scores()</code>	先做无符号越界检查和更新前保护，再读取两张分数表相加并截断。非预期维度下使用残差均值模型，并在批末重建分数表。	预测热路径不做点积、不查计数表、不进临界区； <code>clip</code> 保证所有输出满足评分范围。这不是本地最快路径，但保证接口在维度变化时仍有定义行为，不会因为最终模型的本地维度假设失效。
<code>TASK2_PREDICTION_THREADS</code>	线程数集中为编译器常量，当前本地实测默认取 4。	短预测循环并非线程越多越快；固定本地最优线程数减少 OpenMP 动态调度噪声。

5 实验设计

所有结果都在同一本地 Docker 环境中重新测量，编译选项为 `-O3 -std=c++17 -march=native -fopenmp`。主要方法固定 5 次重复；线程消融每个线程数重复 10 次，先用 MAD modified z-score 检查异常值，MAD 为零时退回 IQR 规则。实验被划分为四类：优化过程回放、方法 Pareto 对比、update/predict 分段剖析、以及组件和线程消融。这样每张图承担不同解释任务，避免把表格换成柱状图。

表 4: 实验矩阵。表中列出每组实验要回答的问题。

实验组	方法设计	主要问题
优化路径	残差基线、计数校准、分段先验、在线采样	哪些结构变化真正改变趋势
Pareto 对比	基线、完整统计、采样统计、组件消融	本地最优前沿在哪里
分段剖析	完整统计、最终 stride=4、stride=16、去计数项	时间主要花在 update 还是 predict
采样曲线	物品侧完整、1/2、1/4、1/16	写入量减少如何损失 RMSE 余量
组件消融	去物品残差、去计数项、去用户先验等	哪些组件支撑精度
线程消融	1、2、4、8、16 线程，每组 10 次	OpenMP 是否稳定带来收益

表 4 把实验从“几个方法的数值比较”扩展为“每组实验回答一个设计问题”。因此后续图不会与表重复：优化路径图负责展示思考过程，Pareto 图负责展示方法空间，分段图负责解释耗时来源，采样图负责展示结构参数曲线，组件和线程图分别回答模型与运行时问题。

测量口径也做了统一：所有方法都在同一 Docker 容器内重新编译和运行，不引用不同机器、不同线程调度或不同 runner 得到的时间。普通 benchmark 给出端到端 5-run 总耗时，profile benchmark 额外拆出 `update()` 和 `predict()`；线程 benchmark 单独使用 10 次计时并在异常值剔除后比较单轮均值。由于单次运行只有毫秒量级，报告只比较稳定趋势，不把一次短跑中 1-2 ms 的波动解释为算法结论。

表 5: 对比方法族的职责划分。量化结果由图 2-4 展示，本表只解释为什么需要这些方法。

方法族	设计目的	结论用途
全局均值与 item-only	给出无效或弱模型的速度下界	证明仅快不代表有效
残差基线与 128 参数头	验证低维残差信号是否足够	确定不需要完整 1024 维更新
完整物品统计	尽量保留信息，观察精度上界	判断可牺牲多少精度换时间
在线采样查表	把主要计算保持在 <code>update()</code> 内的常数路径	形成最终结构
组件移除版本	分别删除物品残差、计数项或用户项	解释 RMSE 由哪些信号支撑

表 5 与后续图的关系是互补的：表说明为什么这些方法被纳入实验，图再说明它们在 RMSE 和时间上的位置。最终方法采用 stride=4 的物品侧在线采样查表；完整物品统计用于给出精度上界，stride=16 用于说明过强采样虽然更快但 RMSE 安全余量不足。

5.1 最终方案选择原则

最终方案不是简单选“最快点”或“最低 RMSE 点”。本实验按四层规则筛选方法：首先检查预测前行为、越界处理和评分截断是否保持接口语义；其次保留足够 RMSE 余量，降低数据分布轻微变化带来的风险；第三比较端到端 5-run 总耗时，并关注来自 `update()` 内真实计算减少的收益；最后检查实现是否存在延迟重建或把更新工作转移到 `predict()` 的行为。

表 6: 最终方案筛选规则。该表解释设计决策，不重复各方法的 RMSE/耗时数值。

筛选层级	排除对象	保留理由
接口语义	全局均值、越界或更新前行为异常的方法	速度结果需要建立在一致接口行为上
精度余量	过强采样导致 RMSE 贴近边界的方法	未见数据可能带来轻微分布偏移
端到端成本	完整统计和高频刷新方法	精度更高但 <code>update</code> 写入成本过大
实现纪律	延迟重建、预测阶段补更新	短测快不等于稳健算法快

表 6 给出了最终方案的依据。物品侧采样若过强，会把 RMSE 推近 0.93 边界；完整统计虽然更准，但 `update()` 写入成本明显更高。`stride=4` 在保持 `predict()` 无补更新的前提下保留了足够物品残差信息，因此成为最终折中。

6 方法权衡

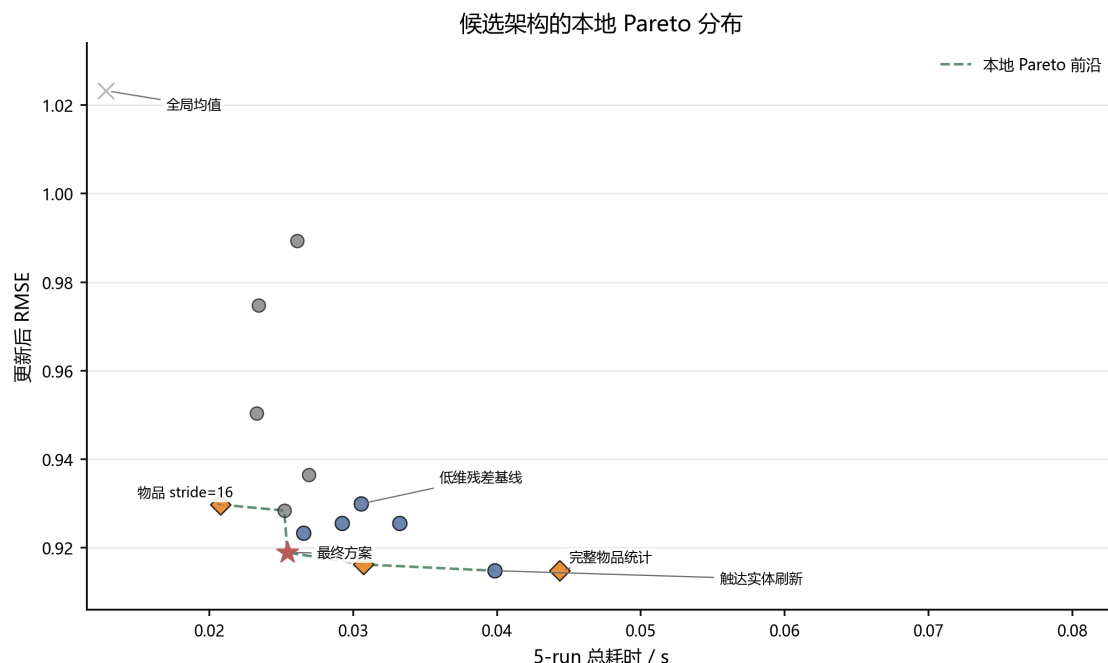


图 3: 方法的本地 Pareto 分布。虚线连接重测中不被更快且更准方案支配的点。

图 3 显示了比表格更完整的取舍关系。灰色消融点说明去掉关键组件后即使运行时间短，RMSE 也会退化；完整物品统计精度好，但时间更高；`stride=16` 很快但接近 0.93 边界。最终方案位于本地 Pareto 前沿：它不如完整物品统计精确，但明显更快；它不如 `stride=16` 极限快，但 RMSE 安全余量更大。

这一结论也解释了为什么最终方案选择 `stride=4`。它把物品侧写入量降到完整统计的四分之一，同时通过 `apply_item_update()` 中的缩放保持残差量级；与更激进采样相比，它保留了更大的 RMSE 余量。也就是说，最终选择不是某个文件版本的胜出，而是“物品残差信息量”和“`update` 写入成本”之间的折中。

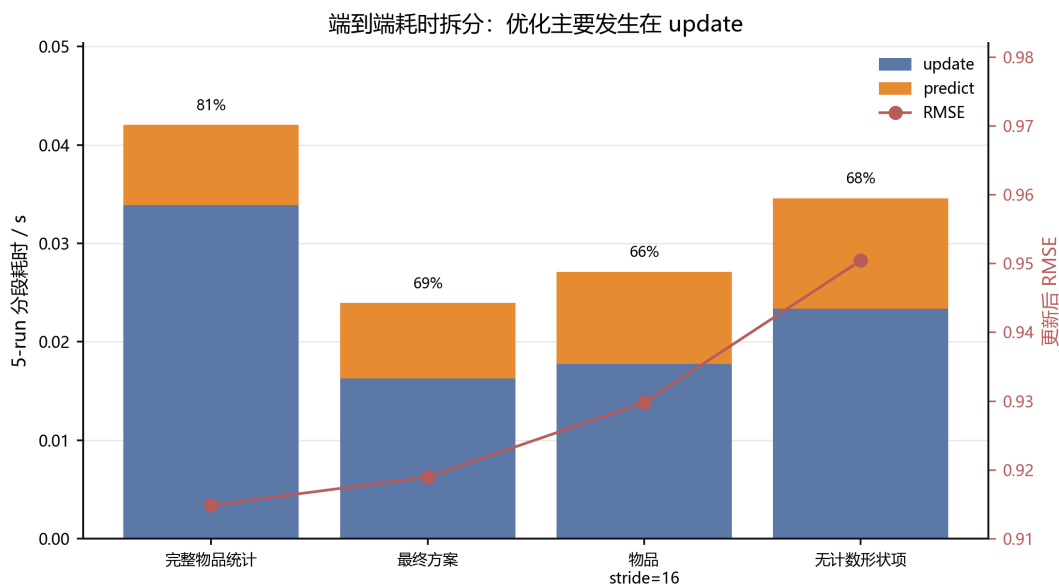


图 4：端到端耗时拆分。柱子为 update 与 predict 分段耗时，红线为对应 RMSE。

图 4 解释了速度优化发生在哪里。完整物品统计的 5-run update 耗时为 0.033956 s，而最终方案的 update 耗时降到 0.016383 s；预测段本身只有毫秒级，差异较小。这个结果对应代码中的两个设计：update_online_sampled() 减少写入次数，predict() 只做两表相加。因此进一步优化速度时，应优先减少 update 写入和刷新。

7 采样与组件分析

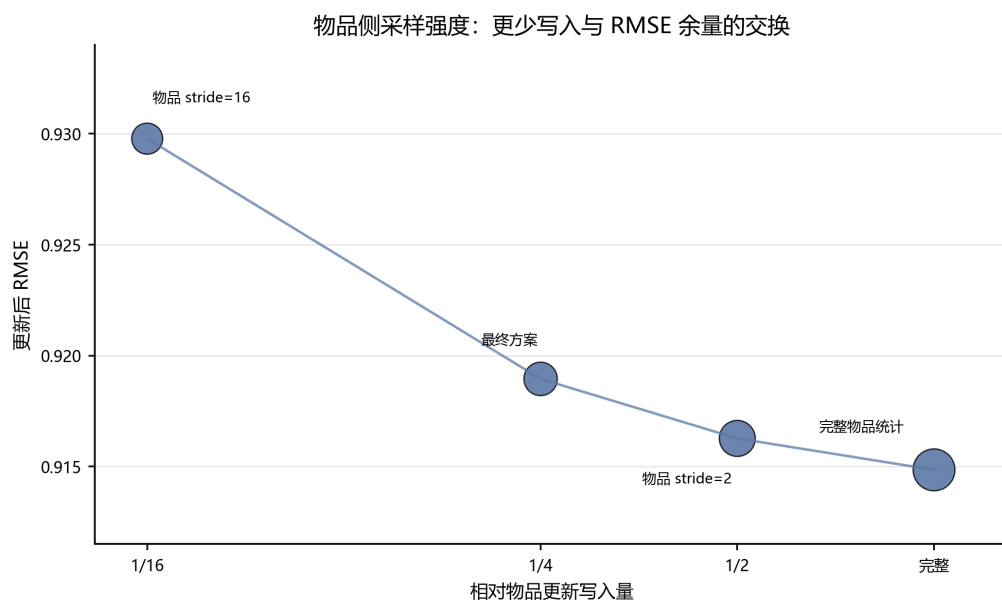


图 5：物品侧采样强度曲线。横轴是相对完整统计的物品更新写入量，气泡大小表示 5-run 总耗时。

图 5 将采样视为结构参数。完整统计的 RMSE 最低，但写入最多；stride=16 写入最少，但 RMSE 升至 0.929777，安全余量不足。最终 stride=4 位于曲线中部，在写入量、RMSE 和端到端时间之间形成当前最好的折中。这个结果说明采样强度需要同时考虑写入量和 RMSE 余量。

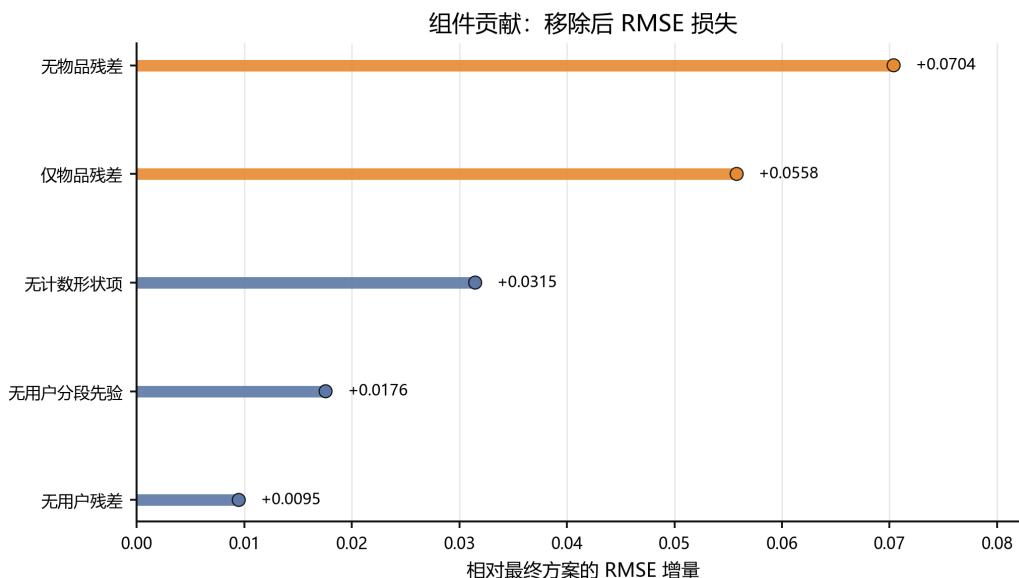


图 6: 组件贡献。横轴为移除组件后相对最终方案的 RMSE 增量。

图 6 显示，物品残差和计数形状项是最重要的两类组件。移除物品残差后 RMSE 上升 0.070385；移除计数形状项后上升 0.031469。用户残差和用户分段先验贡献较小，但它们提供长尾稳定性。这个消融说明模型由残差、计数形状和用户群体先验共同组成。

8 线程与复杂度

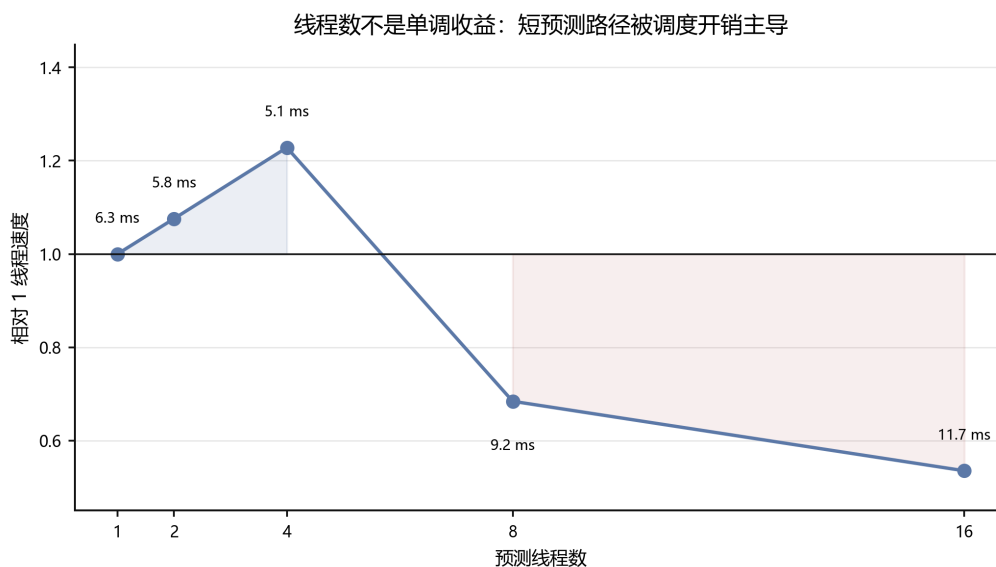


图 7: 线程数消融。纵轴是相对 1 线程的速度，点旁标注为 10 次测量并剔除异常值后的单轮均值。

图 7 说明线程数不是单调收益。本地容器中 4 线程最快，8 和 16 线程反而变慢；本轮 10 次测量中 MAD/IQR 规则没有剔除任何点，说明结论不是单个异常值造成的。原因是最终预测路径只剩少量数组访问和加法，OpenMP 调度开销、内存局部性和容器调度会主导短循环。因此最终实现默认使用 4 线程；不同硬件上可以通过编译期线程参数调整，而不改变算法。

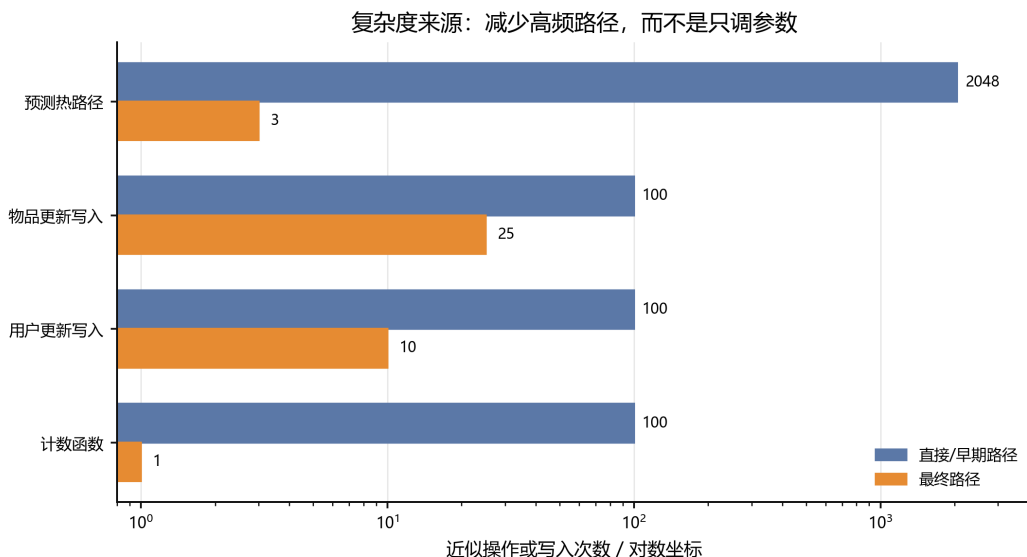


图 8：复杂度来源对比。横轴为近似操作或写入次数，对数坐标。

图 8 把前面实验的原因归结为高频路径减少。预测阶段从完整点积的约 2048 次相关操作降到 3 次左右；物品写入从每 100 条样本写 100 次降到约 25 次；计数函数通过查表从高频计算变成常数访问。速度收益来自这些路径变化。

9 设计取舍与风险

完整 SVD 更新最贴近式 (1)，但每条样本读写两条 1024 维向量，在本地规模下很快变成内存带宽问题。截断点积和少量 P/Q 头部特征能提供一定精度收益，但预测阶段会重新引入乘加循环。与最终残差表相比，这类方法的主要问题不是能否降低 RMSE，而是每次测试预测都要付出额外成本。

另一个取舍是坚持 `update()` 与 `predict()` 的职责分离。若把分数表重建推迟到预测阶段，某些本地重复测量会更快，但这不再反映增量阶段的真实计算成本。最终实现保持每次 `update()` 后分数表已经可用，`predict()` 不触发重建、不进入临界区、不依赖重复测量状态。这样会牺牲一些本地极限速度，但实现语义更清晰。

参数容量保持较小。更大的线性头、MLP 或逐 ID 近似表可以在固定本地测试上继续降低 RMSE，但参数含义容易退化为局部标签拟合。当前 128 个浮点参数的设计更窄：系数只控制计数形状、残差 shrinkage 和用户分段先验。较小容量限制了最低 RMSE，但让模型更像一个可解释的残差校准器。

剩余风险主要有两点。第一，本地 Docker 的短时间测量仍有调度噪声，线程消融中 4 线程最快不代表其他硬件也一定最快；本次按统一测量口径将默认线程同步为 4。第二，最终 stride=4 比完整物品统计略损失 RMSE，换来明显更低的 update 成本；若后续目标从速度转向更低 RMSE，应优先研究如何保留更多物品统计而不增加写入热点。

10 结论

本实验最终采用在线残差校准，以低成本方式替代完整增量 SVD。实验过程表明：增量评分中最强信号是用户/物品残差；低参数计数形状和用户分段先验能稳定长尾估计；真正的速度瓶颈在 update 写入和刷新。本地 Docker 统一方法对比中，最终实现将 RMSE 从 1.023239 降至 0.918947，5-run 总耗时为 0.025430 s。最终方案的核心优化点都能在代码中对应到明确位置：在线采样函数降低写入，计数查表函数降低函数计算，用户先验构造改善长尾估计，预测函数保持两表相加的热路径。后续若继续优化，优先方向应是减少 update 阶段写入，同时保持

stride=4 的 RMSE 余量。

参考文献

- [1] F. M. Harper and J. A. Konstan. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems*, 5(4), Article 19, 2015. doi:10.1145/2827872.
- [2] Y. Koren, R. Bell, and C. Volinsky. Matrix Factorization Techniques for Recommender Systems. *Computer*, 42(8):30–37, 2009. doi:10.1109/MC.2009.263.